NPTEL Online Certification Course
Applied Accelerated Artificial Intelligence
Assignment 1: Detailed Solution
Indian Institute of Technology Guwahati

Question 1:

What is Memory Wall?

- A) The high latency difference between DRAM & CPU
- B) The lack of storage space for large files in secondary hard drives
- C) The physical limitation of the silicon die size in processors
- D) Speed of L1 cache is around 4 cycles per core/private

Correct Answer: A) The high latency difference between DRAM & CPU

Detailed Solution: The "Memory Wall" refers to the growing performance gap between how fast a CPU can execute instructions and how fast data can be transferred from main memory (DRAM). While processor speeds have increased exponentially over decades, memory access times have improved at a much slower rate, causing the CPU to frequently sit idle while waiting for data.

Question 2

What is Re-Order buffer?

- A) Stores recently used pages
- B) Looks ahead for dependent & independent instructions in advance
- C) Stores the recently used instructions
- D) Used in In-Order Execution

Correct Answer: B) Looks ahead for dependent & independent instructions in advance

Detailed Solution: The Re-Order Buffer (ROB) is a critical component used in Out-of-Order (OoO) execution processors. It tracks instructions that are executed out of chronological order due to structural or data dependencies, allowing the processor to safely check ahead for independent instructions while ensuring they are committed (retired) in the correct, original program order.

Question 3

For data level parallelism, which type of extensions are used in Modern systems?

Indian Institute of Technology Guwahati

- A) SIMD
- B) SISD
- C) MISD
- D) MIMD

Correct Answer: A) SIMD

Detailed Solution: SIMD stands for Single Instruction, Multiple Data. It allows a processor architecture to apply a single machine instruction across a vector of multiple data points simultaneously. Modern examples include Intel AVX/AVX-512 and ARM Neon, which are fundamental for accelerating vector/matrix mathematics.

Question 4

The left & right side of intersection of peak performance curve & peak memory bandwidth curve represents?

- A) Memory bound & compute intensive workload
- B) Compute intensive & memory bound workload
- C) Cannot differentiate
- D) None of the above

Correct Answer: B) Compute intensive & memory bound workload

Detailed Solution: This question refers to the Roofline Performance Model. The point of intersection represents the hardware's machine balance. To the left of this inflection point, the workload's performance is limited by the memory bandwidth (Memory-bound). To the right of this point, the performance hits the hardware plateau and is limited by raw processing speed (Compute-bound / Compute-intensive).

Question 5

CPU cannot do transformation attention (large matrix multiplication). Why?

- A) Because it does not fall under the memory roof for CPUs only.
- B) The statement is false, it can be done by CPU.
- C) Because it is above the maximum compute performance of CPU
- D) It is neither done by CPU not by GPU

Correct Answer: C) Because it is above the maximum compute performance of CPU

Detailed Solution: Large matrix operations like Transformer Attention mechanisms require trillions of calculations per second (FLOPs). While a CPU is *technically* capable of running the code logic, its physical architectural limits (fewer cores, lack of specialized tensor processing units) mean these operations require compute performance levels that vastly exceed what a standard CPU can physically deliver in a reasonable timeframe.

Indian Institute of Technology Guwahati

Question 6

LLM inference & DNN training are:

- A) Memory Bound & Compute Bound
- B) Compute Bound & Memory Bound
- C) Memory Bound & Memory Bound
- D) Compute Bound & Compute Bound

Correct Answer: A) Memory Bound & Compute Bound

Detailed Solution: 11

LLM Inference is primarily Memory-bound because it processes tokens sequentially (batch size is often small), requiring the entire set of model weights to be fetched from memory for every single token generated.

DNN Training is primarily Compute-bound because it processes large batch sizes over complex forward and backward passes, maximizing data reuse inside processor registers/caches.

Question 7

DDR5 Memory have:

- A) 2500 - 6400 MT/s, 32 bit wide bus
- B) 4800 - 6400 MT/s, 32 bit wide bus
- C) 1200 - 4800 MT/s, 64 bit wide bus
- D) 4800 - 6400 MT/s, 64 bit wide bus

Correct Answer: D) 4800 - 6400 MT/s, 64 bit wide bus

Detailed Solution: Standard JEDEC DDR5 memory modules operate within baseline speeds of 4800 to 6400 MT/s (MegaTransfers per second). A standard single-channel desktop/server DDR5 DIMM maintains a total data bus width of 64 bits (architecturally split into two independent 32-bit subchannels to improve efficiency).

Question 8

ECC (Error correcting code) Memory in AI servers do:

- A) Detect and correct single bit and double bit errors
- B) Detect single bit errors, correct double bit errors
- C) Detect double bit errors, correct single bit errors
- D) Correct both single bit and double bit errors

Correct Answer: C) Detect double bit errors, correct single bit errors

Detailed Solution: Standard enterprise ECC memory uses SECDED (Single Error Correction,

Indian Institute of Technology Guwahati

Double Error Detection) algorithms. It can seamlessly identify and correct a single-bit corruption on the fly without system interruption, and safely detect (but not fix) a double-bit memory error, triggering a system alert to prevent data corruption.

Question 9

In NUMA architecture (like DGX H100), accessing cross socket memory adds around:

- A) 2x latency
- B) 3x latency
- C) 4x latency
- D) 6x latency

Correct Answer: A) 2x latency

Detailed Solution: In Non-Uniform Memory Access (NUMA) systems, a CPU or GPU has "local" memory directly wired to its socket. If a processor must access memory wired to a *different* socket (cross-socket communication), the data must traverse interconnect fabrics (like UPI or Infinity Fabric), which adds an overhead penalty of approximately 2x latency compared to a direct local fetch.

Question 10

Tick the correct option regarding warps in CUDA:

- A) Warps is 32 threads executing same instruction on multiple data
- B) Warps is 64 threads executing different instruction on multiple data
- C) Warps is 32 threads executing same instruction on single data
- D) Warps is 64 threads executing same instruction on multiple data

Correct Answer: A) Warps is 32 threads executing same instruction on multiple data

Detailed Solution: In NVIDIA's CUDA programming model, a "Warp" is the basic unit of execution. It consists of a group of 32 threads that are executed concurrently in a SIMT (Single Instruction, Multiple Threads) fashion, meaning all 32 threads execute the exact same instruction at any given clock cycle across different pieces of data.

Question 11

The precision in Tensor cores is _____ to TFLOPS.

- A) Exponentially increasing
- B) Exponentially decreasing
- C) Inversely proportional
- D) Directly proportional

Correct Answer: C) Inversely proportional

Indian Institute of Technology Guwahati

Detailed Solution: As numerical precision decreases (e.g., shifting down from FP32 to FP16, BF16, or INT8), the processing hardware can handle far more calculations simultaneously within the same physical silicon footprint. Therefore, lower numerical precision leads to higher raw processing throughput (TFLOPS), making precision and performance throughput inversely proportional.

Question 12

" torch.cuda.amp.autocast() " is used to:

- A) Default to higher precision units
- B) Default to lower precision units
- C) Automatic Mixed Precision
- D) None of the above

Correct Answer: C) Automatic Mixed Precision

Detailed Solution: torch.cuda.amp.autocast() is PyTorch's utility handler for Automatic Mixed Precision (AMP) training. It automatically matches specific operations (like matrix multiplications) to faster, lower-precision data types (like float16/bfloat16) while keeping critical operations (like loss scaling and reductions) in float32 to preserve model convergence.

Question 13

Which type of advantage does the HBM3 have over H100 & AMD MI300X?

- A) Computation advantage
- B) Memory storage advantage
- C) Multi core CPU advantage
- D) Memory Bandwidth advantage

Correct Answer: D) Memory Bandwidth advantage

Detailed Solution: High Bandwidth Memory 3 (HBM3) utilizes a 3D-stacked DRAM design connected via an ultra-wide silicon interposer interface. This provides massive memory bandwidth advantages (reaching terabytes per second) compared to traditional GDDR or DDR memory subsystems, keeping hyper-parallel computing engines properly saturated with data.

Question 14

What is the primary architectural bottleneck that NVIDIA's NVSwitch is designed to resolve?

- A) The storage read/write limitations between local NVMe drives and system memory.
- B) The bandwidth and scalability limits of point-to-point NVLink connections across multiple GPUs.
- C) The latency delays caused by PCIe slot power throttling during deep learning workloads.

Indian Institute of Technology Guwahati

- D) The network congestion between different server racks inside a data center over standard Ethernet.

Correct Answer: B) The bandwidth and scalability limits of point-to-point NVLink connections across multiple GPUs.

Detailed Solution: While NVLink allows direct high-speed point-to-point connections between a limited number of local GPUs, scaling beyond a single baseboard requires a centralized crossbar system. NVSwitch enables multiple NVLink links to map across a unified switching fabric, allowing all GPUs inside an enterprise node (or across multiple nodes) to communicate at full NVLink speeds simultaneously.

Question 15

Which of the following functions is NOT a CUDA Runtime API function?

- A) cudaFree
- B) cudaMemcpy()
- C) cudaMalloc()
- D) cuMemAlloc()

Correct Answer: D) cuMemAlloc()

Detailed Solution: Functions prefixing with lower-case cuda (such as cudaMalloc, cudaFree, cudaMemcpy) belong to the higher-level CUDA Runtime API. Functions prefixing with cu and utilizing alternative casing (such as cuMemAlloc) belong directly to the lower-level CUDA Driver API, which interacts closer to the hardware level.

Indian Institute of Technology Guwahati

NPTEL Online Certification Course
Applied Accelerated Artificial Architecture
Assignment 2 : Detailed Solution
Indian Institute of Technology Guwahati

Question 1:
Which of the following best describes the key structural difference between a Docker Image and a Container?

Options:
A. An image consists of a mutable filesystem, whereas a container is an immutable snapshot that cannot be modified.
B. An image contains multiple layers and metadata, while a container is an instance of that image which adds its own writable layer, namespaces, and cgroups.
C. A container is built exactly once using a Dockerfile step, whereas an image can be dynamically scaled using cgroups.
D. Images use shared filesystem diffs to run, while containers completely copy the entire 5–10 GB base filesystem to run independently.

Correct Answer: B
Explanation:
A Docker image is a read-only, immutable template made up of stacked filesystem layers and config metadata. A container is a runnable instance of that image. When a container starts, the Docker engine adds a thin, temporary writable layer (container layer) on top of the image stacks and isolates the process using Linux namespaces (for environment isolation) and cgroups (for resource limiting).

Question 2 :
Why are layer caching and the ordering of steps considered critical when building large AI images?

Options:
A. Layer caching prevents the container's writable layer from exceeding the 5–10 GB limit.
B. Incorrect step ordering deletes the underlying image metadata, forcing the runtime to crash.
C. They drastically reduce rebuild times (e.g., from 30 minutes to 30 seconds) by reusing unchanged filesystem diffs shared between images.

Indian Institute of Technology Guwahati

D. They turn a stopped container instance back into an immutable image.
Correct Answer: C
Explanation:
Docker builds images layer-by-layer. If a layer's contents and instructions have not changed, Docker reuses the cached layer. However, if a step changes (like modifying local application code), that cache and all subsequent layer caches are invalidated. Putting slow-changing, heavy steps (like installing CUDA packages or python dependencies) *before* fast-changing steps (like copying application source code) ensures that heavy installations are cached, dramatically speeding up subsequent builds.

Question 3:
The driver version must be old enough for the toolkit in the image.

Options:
A. TRUE
B. FALSE

Correct Answer: B
Explanation:
This statement is false. The NVIDIA host driver (kernel-space driver installed on the host operating system) must be new enough to support the CUDA driver API version required by the CUDA toolkit/libraries packaged inside the container image. Host drivers are generally backward-compatible, meaning newer host drivers can run containers with older CUDA toolkits, but old host drivers cannot run newer CUDA toolkit versions.

Question 4 :
Does Docker give us control to run any program on any specific available GPU?

Options:
A. TRUE
B. FALSE

Correct Answer: A
Explanation:
Docker, in tandem with the NVIDIA Container Toolkit, gives you granular control over hardware access. You can isolate and target specific GPUs for a container using flags such as `--gpus '"device=0"'` or by passing the `NVIDIA_VISIBLE_DEVICES` environment variable, restricting the containerized application's visibility to only the selected physical GPUs.

Indian Institute of Technology Guwahati

Question 5 :
What is the correct command to install the NVIDIA Container Toolkit?

Options:
A. `sudo apt-get install -y nvidia-container-toolkit`
B. `sudo install -y nvidia-container-toolkit`
C. `sudo apt-get install -y nvidia-containertoolkit`
D. `sudo apt-get install -a nvidia-container-toolkits`

Correct Answer: A
Explanation:
The correct package name in the official NVIDIA apt repositories is `nvidia-container-toolkit`. Using the standard Debian/Ubuntu package manager `apt-get` with the `-y` (assume-yes) flag is the standard and correct syntax to install the utility.

Question 6 :
A Docker reproducibility bug occurs when local code hasn't changed, but "something changed in the resolver's world." Which scenario best exemplifies this?

Options:
A. A non-deterministic race condition triggers in the application code.
B. A corrupted local Docker cache forces the engine to skip build layers.
C. An unpinned dependency or base image tag pulls a newer, breaking version during a fresh build.
D. A host OS kernel update changes how the container handles low-level resource limits.

Correct Answer: C
Explanation:
If a Dockerfile points to a dynamic tag (like `ubuntu:latest` or `python:3.10`) or installs packages without strict version locking (like `pip install requirements.txt` with unpinned packages), a rebuild at a later date will query external registries. If an upstream maintainer updates that tag or package with breaking changes, your build will fail or break despite your local application code remaining completely identical.

Question 7 :
Even after pinning human-readable tags, Docker builds can still suffer from reproducibility bugs. Which technique guarantees byte-level build reproducibility?

Options:
A. Using semantic version ranges (e.g., `>=3.1.5`) in the Dockerfile.

Indian Institute of Technology Guwahati

B. Enabling strict tag-locking in the host daemon settings.
C. Appending a `.lock` extension to the target build image name.
D. Referencing the base image by its immutable SHA-256 digest.

Correct Answer: D
Explanation:
Human-readable tags (like `python:3.9-slim`) are mutable—they can be updated by registry owners to include patches or changes. To guarantee absolute, byte-for-byte reproducibility, you must reference the unique, cryptographic SHA-256 hash of the image manifest (e.g., `python:3.9-slim@sha256:...`). Because cryptographic digests are immutable, the engine will always pull the exact same set of image bytes.

Question 8 :
When using `COPY --from=builder`, what happens to the layers of the `builder` stage in the final image?

Options:
A. They are stored as metadata but don't take up disk space.
B. They are compressed and hidden within the final layer.
C. They are completely excluded from the final image's manifest.
D. They become read-only parent layers for the production stage.

Correct Answer: C
Explanation:
In a multi-stage build, each `FROM` instruction begins a brand new stage with a clean filesystem. When you use `COPY --from=builder`, you are selectively picking out only the specified artifacts/files from that prior stage. The intermediate layers created during the `builder` stage (such as compiler dependencies, build tools, and temporary caches) are discarded entirely and are not referenced in the final image's manifest, resulting in a lean production image.

Question 9 :
What is the correct command to generate a reproducible `requirements.txt` file with dependency hashes using the `pip-tools` suite?

Options:
A. `pip-compile --generate-hashes --output-file=requirements.txt requirements.in`
B. `pip-compile --lock-dependencies --export=requirements.txt requirements.in`
C. `pip-compile --freeze-hashes --output requirements.txt requirements.in`
D. `pip-compile --strict-verification --write-to requirements.txt requirements.in`

Indian Institute of Technology Guwahati

Correct Answer: A
Explanation:
The official CLI tool `pip-compile` uses the `--generate-hashes` flag to parse input requirements (such as `requirements.in`), resolve dependencies, and download and write the security SHA-256 hashes for each resolved package. The `--output-file=` option dictates where the fully locked dependencies file should be written.

Question 10 :
What is the Docker mechanism called when you link a specific, absolute path on your host hard disk directly to a path inside a container to persist your files even after the container is stopped?
Options:
A. Docker Cache Layers
B. Host Port Mapping
C. Container Image Commits
D. Bind Mounts

Correct Answer: D
Explanation:
A Bind Mount maps a user-specified, absolute path from the host computer directly to a mount point inside the container. Because it references the host filesystem, any modifications made within the container write directly to your local hard drive. This keeps your physical files intact, editable, and saved even if the container is stopped, deleted, or rebuilt.

Question 11 :
When using Unified Memory in CUDA, how does `cudaMallocManaged` handle memory management between the CPU and the GPU?

Options:
A. It allocates a single pointer accessible by both the CPU and GPU, automatically migrating data between them as needed.
B. It creates two completely different pointers that require manual synchronization functions after every operation.
C. It locks the pointer so only the CPU can read the data while the GPU is actively executing a kernel.
D. It duplicates the pointer across the physical hard drives of both the host system and the graphics card.

Correct Answer: A

Indian Institute of Technology Guwahati

Explanation:
`cudaMallocManaged` allocates Unified Memory, which returns a single pointer that can be referenced by both CPU and GPU code. The CUDA driver and system software automatically handle page faults and migrate data between the system RAM (CPU) and video memory (GPU) on demand, removing the need for developers to write manual data transfer commands like `cudaMemcpy`.

Question 12 :
When launching a CUDA kernel using the execution configuration syntax `kernel<<<X, Y>>>`, what do the parameters `X` and `Y` dictate to the GPU hardware?

Options:
A. `X` is the number of thread blocks in the grid, and `Y` is the number of parallel threads within each block.
B. `X` is the number of threads per block, and `Y` is the number of thread blocks in the grid.
C. `X` is the amount of allocated constant memory, and `Y` is the amount of allocated shared memory.
D. `X` is the maximum loop count per thread, and `Y` is the number of physical GPU streaming multiprocessors (SMs) to engage.

Correct Answer: A
Explanation:
In CUDA's triple angle bracket launch configuration `<<<gridDim, blockDim>>>`:

- The first parameter (`X`) represents the dimension of the grid, which specifies the total number of thread blocks to spawn.
- The second parameter (`Y`) represents the dimension of a block, which specifies the number of threads inside each individual thread block.

Question 13 :
Which GPU sharing mechanism provides strict hardware-level isolation, ensuring that dedicated memory and SMs are completely partitioned so one tenant cannot trigger an Out-of-Memory (OOM) error in another?

Options:
A. Time-slicing
B. Multi-Process Service (MPS)
C. Multi-Instance GPU (MIG)
D. Dynamic Core Over-provisioning
Correct Answer: C

Explanation:

Indian Institute of Technology Guwahati

Multi-Instance GPU (MIG) splits a single physical GPU into several separate, independent hardware instances (up to 7 on architectures like the H100 or B200). Each partition has dedicated, isolated memory, memory controllers, and Streaming Multiprocessors (SMs). This strict hardware separation ensures that high resource usage or an OOM crash in one instance has absolutely no effect on neighboring instances.

Question 14 :
What is a major isolation limitation when using CUDA Multi-Process Service (MPS) to share a GPU between different processes?

Options:
A. It completely stops multiple processes from submitting kernels at the same time.
B. Memory is still shared, meaning a buggy process can corrupt another's memory or crash the entire MPS server.
C. It drops small inference request throughput by 30-50%.
D. It enforces strict hardware-level partitions that prevent dynamic memory allocation entirely.

Correct Answer: B
Explanation:
Unlike MIG, MPS uses logical partitioning (soft isolation). Multiple processes feed commands to a single control server to share SM hardware concurrently. However, the physical memory space is not strictly partitioned in hardware. Therefore, a memory access bug or an OOM crash in one client process can corrupt the address space of other active processes or crash the central MPS server daemon, taking all sibling client processes down with it.

Question 15 :
How does gang scheduling solve the "stranded GPU" problem in multi-node clusters?

Options:
A. It kills running idle nodes if they take too long to start.
B. It splits a large 4-node job into smaller, single-node jobs.
C. It ensures all required nodes launch simultaneously, allowing no partial allocations.
D. It automatically downloads missing training data to the cluster.

Correct Answer: C
Explanation:

Indian Institute of Technology Guwahati

In distributed training across multiple nodes, a job cannot run unless all nodes are online and synchronized. The "stranded GPU" issue occurs when some nodes are allocated and sit idle, locking up precious resources while waiting on a missing node that is still queued. Gang scheduling (as used in schedulers like Slurm) avoids this by making an "all-or-nothing" decision: it only runs the job when every single required node is simultaneously ready, eliminating wasteful partial allocations.

Indian Institute of Technology Guwahati

NPTEL Online Certification Course
Applied Accelerated Artificial Architecture
Assignment 3 : Detailed Solution
Indian Institute of Technology Guwahati

Question 1

In Container Orchestration, which phase of the orchestration process involves continuously comparing the user's declared target configuration (desired state) against what is currently running (actual state)?

- A) You declare
- B) IT compiles
- C) It reconciles
- D) It watches

Correct Answer: D) It watches

Explanation: The "watch" phase continuously monitors the current cluster state to detect drift from the desired state. This streaming observation triggers reconciliation loops to restore target configurations automatically.

Question 2

Which internal Google system served as the original foundation for Kubernetes before it was open-sourced in 2014 and donated to the CNCF in 2015?

- A) Kuberflow
- B) Helm
- C) Borg
- D) Argo

Correct Answer: C) Borg

Explanation: Borg was Google's internal large-scale cluster management system built to manage millions of jobs. Kubernetes was designed as an open-source rewrite of Borg, drawing on years of operational lessons learned.

Question 3

In the Kubernetes control plane architecture, which component acts as "the only door in" by handling authentication, validation, and admission control?

- **A) etcd**
- **B) API Server**

Indian Institute of Technology Guwahati

- **C) Scheduler**
- **D) Controller Manager**

Correct Answer: B) API Server

Explanation: The kube-apiserver acts as the central hub and primary entry point for all control plane interactions. It authenticates, authorizes, and validates all incoming REST requests before storing state changes in etcd.

Question 4

Which component on a Kubernetes Worker Node is responsible for configuring iptables or eBPF rules to ensure network traffic for Services properly routes to live Pod IPs?

- A) kubelet
- B) kube-proxy
- C) containerd
- D) NVIDIA device plugin

Correct Answer: B) kube-proxy

Explanation: kube-proxy runs on every node to manage network routing rules via technologies like iptables or IPVS/eBPF. It proxies virtual Service IP traffic directly to the dynamically changing underlying Pod network endpoints.

Question 5

Which of the following correctly describes how storage volumes and IP addresses are assigned to containers within a Pod?

- A) Each container gets its own IP address, but volumes cannot be shared across containers.
- B) A single Pod IP is shared by all containers in the Pod, and volumes can mount into any container in the Pod.
- C) Volumes mount exclusively to the main application container and are inaccessible to sidecar containers.
- D) IP addresses are assigned at the container level, while storage volumes are assigned at the node level.

Correct Answer: B) A single Pod IP is shared by all containers in the Pod, and volumes can mount into any container in the Pod.

Explanation: Pods serve as the atomic unit of scheduling where all co-located containers share a single network namespace (and IP). Mounted storage volumes are shared across containers in the same Pod via local mount points.

Indian Institute of Technology Guwahati

Question 6

Which Kubernetes component is required to advertise a GPU as an extended resource to the scheduler?

- A) Docker Desktop Virtual Machine
- B) Nvidia Device Plug-in
- C) WSL2 Ubuntu Host
- D) Cube Control Labeling Module

Correct Answer: B) Nvidia Device Plug-in
Explanation: The NVIDIA Device Plugin is a DaemonSet that discovers host GPU hardware and registers GPU capacity with `kubelet`. This enables the Kubernetes scheduler to track and allocate GPUs as extended resources (`[nvidia.com/gpu](https://nvidia.com/gpu)`).

Question 7

What information is explicitly displayed inside the pod logs when running the `gpu-smoke-working.yaml` **program successfully?**

- A) A benchmark script testing floating-point operations
- B) An nvidia-smi table format output
- C) A direct memory dump of the physical GPU VRAM
- D) A continuous stream of CUDA core temperature metrics

Correct Answer: B) An nvidia-smi table format output
Explanation: Smoke tests for GPU access execute a lightweight container invoking `nvidia-smi` to verify hardware access. Success produces the standard tabular output listing GPU model, driver details, and CUDA runtime availability.

Question 8

Why are headless services configured specifically for stateful AI clusters instead of utilizing a default cluster IP address?

- A) They act as an encrypted cluster firewall layer.
- B) They map DNS lookups directly to individual pod IPs.
- C) They load-balance external network data globally.
- D) They alter the core container base settings dynamically.

Correct Answer: B) They map DNS lookups directly to individual pod IPs.
Explanation: Headless Services (specifying `clusterIP: None`) bypass single virtual IP load balancing. They return direct `A`/`AAAA` DNS records for individual Pod IPs, enabling direct point-to-point worker communication in stateful setups.

Indian Institute of Technology Guwahati

Question 9

Which operational practice must be followed when managing sensitive access tokens like Hugging Face (HF) API keys?

- A) Store the tokens directly within plain-text configuration maps.
- B) Bake authorization keys straight into the Dockerfile source layer.
- C) Isolate and hold the sensitive data using Kubernetes Secrets.
- D) Host credentials on a public cloud load-balancing endpoint.

Correct Answer: C) Isolate and hold the sensitive data using Kubernetes Secrets.
Explanation: Hardcoding sensitive tokens into images or ConfigMaps exposes them to leakages and source control risks. Kubernetes Secrets decouple sensitive credentials, allowing secure mounting at runtime without exposing raw keys.

Question 10

Which directive is used within a container pod's manifest file to import all key-value configurations directly into the container's environment?

- A) dataFrom
- B) configMapRef
- C) envFrom
- D) valueFrom

Correct Answer: C) envFrom
Explanation: `envFrom` bulk-injects all key-value pairs from a specified ConfigMap or Secret as container environment variables. This avoids explicitly declaring individual variables using `valueFrom`.

Question 11

If a pod manifest references a Secret named `ml-tokens` via `envFrom`, what occurs if the `create-secret.sh` script has not been run first?

- A) The pod skips the variables and starts anyway.
- B) The cluster dynamically generates empty values.
- C) The pod gets stuck in a creation error state.
- D) The system switches the type to a ConfigMap.

Correct Answer: C) The pod gets stuck in a creation error state.
Explanation: When a Pod references a non-existent Secret or ConfigMap during startup, `kubelet` cannot populate required environment variables. As a result, the Pod fails to start and remains stuck in `CreateContainerConfigError`.

Indian Institute of Technology Guwahati

Question 12

During the initial phase of the Canary rollout demonstration, how is the traffic split configured between the stable version and the canary version?

- A) 50% Stable / 50% Canary
- B) 75% Stable / 25% Canary
- C) 90% Stable / 10% Canary
- D) 100% Stable / 0% Canary

Correct Answer: C) 90% Stable / 10% Canary
Explanation: Canary rollouts mitigate risk by initially exposing new releases to a minimal user slice. A 90/10 split allows safely observing metrics and error logs on 10% canary traffic before committing to a full deployment.

Question 13

What happens to the cluster IP mapping when all worker pods are clubbed together into a regular Virtual IP (VIP) service?

- A) A single, unified Cluster IP is generated for the service.
- B) Each pod continues to use a completely unmapped public IP address.
- C) The control plane crashes due to an IP conflict.
- D) The service shifts exclusively to using external NodePorts.

Correct Answer: A) A single, unified Cluster IP is generated for the service.
Explanation: Standard Kubernetes Services assign an internal virtual IP (ClusterIP) that abstracts underlying Pod pods. Traffic directed to this single IP is automatically load-balanced across all matching endpoints.

Question 14

When verifying the injected configuration inside the training pod, which command structure is used by the speaker to check the active environment variables?

- **A)** `kubectl env-verify pod training-pod`
- **B)** `kubectl exec <pod-name> -- env`
- **C)** `kubectl inspect config --pod`
- **D)** `kubectl get env-form training-pod`

Correct Answer: B) `kubectl exec <pod-name> -- env`
Explanation: Executing `kubectl exec <pod-name> -- env` runs the Linux `env` command directly inside a running container. This dumps all active environment variables, validating that ConfigMaps or Secrets were successfully injected.

Indian Institute of Technology Guwahati

Question 15

What is generated for three stateful set workers when configuring a Headless Service?

- A) A single virtual IP address.
- B) Separate, individual IP addresses for each worker pod.
- C) A public load balancer endpoint.
- D) An egress gateway routing table.

Correct Answer: B) Separate, individual IP addresses for each worker pod.
Explanation: Headless Services forgo allocating a single shared virtual IP. Instead, DNS query responses provide direct, individual IP addresses for each matching Pod in the StatefulSet to support direct inter-node communication.

Indian Institute of Technology Guwahati

NPTEL Online Certification Course
Applied Accelerated Artificial Architecture
Assignment 3 : Detailed Solution
Indian Institute of Technology Guwahati

Question 1:

What is the final shape of the tensor after performing the following tensor operations sequentially?

```
import torch
# Initial tensor
x = torch.randn(2, 1, 3)

# Sequential tensor transformations
y = x.unsqueeze(-1)
z = y.squeeze(dim=0)
w = z.squeeze(dim=-1)
print(w.shape)
```

- A) torch.Size([2, 1, 3])
- B) torch.Size([1, 2, 3, -1])
- C) torch.Size([2, 1, 3, 1])
- D) torch.Size([1, 2, 3])

Correct Answer: A) torch.Size([2, 1, 3])

Explanation: unsqueeze(-1) adds a dimension at the end to make shape (2, 1, 3, 1), but squeeze(dim=0) does nothing because dimension 0 has size 2 (not 1). Finally, squeeze(dim=-1) removes the trailing size-1 dimension, returning the tensor back to its original shape (2, 1, 3).

Question 2

What is the primary technical reason PyTorch recommends avoiding in-place operations on tensors participating in an autograd graph?

Indian Institute of Technology Guwahati

- A) Always
- B) Sometimes
- C) Never
- D) None of them

Correct Answer: C) Never
Explanation: In-place operations directly modify tensor memory, which destroys the original values needed by autograd to compute derivatives during the backward pass. PyTorch disallows or raises errors on these operations to maintain graph integrity and prevent silent, incorrect gradient calculations.

Question 3 :

According to PyTorch's broadcasting rules, how are dimensions compared between two tensors?

- A) From left to right, missing dimensions are padded with zeros.
- B) From right to left (trailing dimensions first), where two dimensions are compatible if they are equal or one of them is 1.
- C) From the largest dimension to the smallest dimension.
- D) PyTorch randomly aligns dimensions until it finds a valid combination.

Correct Answer: B) From right to left (trailing dimensions first), where two dimensions are compatible if they are equal or one of them is 1.
Explanation: Broadcasting aligns tensor shapes by comparing their dimension sizes starting from the rightmost (trailing) dimension and moving leftward. Two dimensions are compatible only if they are equal or if one of them has a size of 1, which gets expanded to match the other.

Question 4 :

What happens to the memory usage of your program when you set `requires_grad=True` for many tensors?

- A) Memory remains exactly same
- B) Python stores the computational graph for backward propagation
- C) Memory usage decreases
- D) Memory cleared automatically

Correct Answer: B) Python stores the computational graph for backward propagation
Explanation: Setting `requires_grad=True` instructs PyTorch to construct an execution graph that tracks every intermediate tensor operation performed. This graph holds references to internal activation state tensors in memory so gradients can be properly computed during `.backward()`.

Indian Institute of Technology Guwahati

Question 5

Which segment / line would cause this PyTorch module implementation to fail and throw an error?

```
import torch
import torch.nn as nn

class MyLayer(nn.Module):
    def __init__(self):
        super().__init__()          # ALWAYS call this
        self.linear = nn.Linear(4, 3) # creates W and b
        self.relu = nn.ReLU

    def forward(self, x):
        x = self.linear(x)          # calls linear.forward(x)
        return self.relu(x)

# Instantiate the layer
layer = MyLayer()

# Pass input tensor through the layer
out = layer(torch.randn(2, 4))      # calls Layer.forward()

print(out)
```

- A) self.relu = nn.ReLU
- B) def forward(self, x):
- C) x = self.linear(x)
- D) return self.relu(x)

Correct Answer: A) self.relu = nn.ReLU

Explanation: The provided snippet is fully syntactically correct and outputs a valid tensor of shape (2, 3) without raising any exceptions. It only fails if option A (self.relu = nn.ReLU without parentheses) is substituted into __init__, making self.relu an uninstantiated class rather than a callable module.

Indian Institute of Technology Guwahati

Question 6

Which of the following correctly describes the flow of execution for a single forward pass in `DynamicNet`?

```
import torch
import torch.nn as nn
class DynamicNet(nn.Module):
    def __init__(self, d):
        super().__init__()
        # Dynamic architecture combining Linear and ReLU modules
        self.layers = nn.ModuleList()
        for _ in range(2):
            self.layers.append(nn.Linear(d, d))
            self.layers.append(nn.ReLU())
    def forward(self, x):
        for layer in self.layers:
            x = layer(x)
        return x

model = DynamicNet(d=4)
```

- A) Linear(4, 4) -> Linear(4, 4) -> ReLU() -> ReLU()
- B) ReLU() -> Linear(4, 4) -> ReLU() -> Linear(4, 4)
- C) Linear(4, 4) -> ReLU() -> Linear(4, 4) -> ReLU()
- D) Linear(4, 4) and ReLU() applied in parallel across two separate streams, then merged.

Correct Answer: C) Linear(4, 4) -> ReLU() -> Linear(4, 4) -> ReLU()

Explanation: The __init__ loop alternates appending a Linear(d, d) layer followed immediately by a ReLU() activation layer twice into nn.ModuleList. In forward(),

Indian Institute of Technology Guwahati

iterating over self.layers executes these four sub-modules strictly sequentially in the exact order they were added.

Question 7

Which of the following statements is true regarding PyTorch loss functions (such as CrossEntropyLoss, BCEWithLogitsLoss, MSELoss, L1Loss, HuberLoss)?

- A) CrossEntropyLoss requires a Softmax layer in the model.
- B) BCEWithLogitsLoss requires integer target labels.
- C) MSELoss is robust against extreme data outliers.
- D) CrossEntropyLoss takes raw logits and integer labels.

Correct Answer: D) CrossEntropyLoss takes raw logits and integer labels.
Explanation: PyTorch's `CrossEntropyLoss` internally combines `LogSoftmax` and `NLLLoss` into a single numerically stable function, so models should output raw logits without a manual Softmax. It expects unnormalized continuous model outputs paired with target class indices formatted as integers.

Question 8

What is the purpose of calling `model.eval()` at the beginning of an evaluation loop?

- A) To clear previously computed gradients.
- B) To enable automatic mixed precision (AMP) during evaluation.
- C) To disable layers like dropout and batch normalization.
- D) To calculate gradients.

Correct Answer: C) To disable layers like dropout and batch normalization.
Explanation: Calling `model.eval()` switches the model into evaluation mode, altering the behavior of specific mechanisms like Dropout and Batch Normalization. It stops Dropout from randomly zeroing out activations and forces Batch Normalization to use running dataset statistics instead of batch statistics.

Question 9

Why is it crucial to call `optimizer.zero_grad()` in the training loop before computing the backward pass?

- A) To update the weight parameters and biases.
- B) To move the data to the target device.
- C) To compute the loss function.
- D) To reset the old gradient values from previous iterations.

Correct Answer: D) To reset the old gradient values from previous iterations.

Indian Institute of Technology, Guwahati

Explanation: By default, PyTorch accumulates (adds together) gradients in `.grad` buffers during every backward pass rather than overwriting them. Calling `optimizer.zero_grad()` clears these stale values so old gradients from previous mini-batches do not corrupt current weight updates.

Question 10

In PyTorch training pipelines, what error occurs if data is not moved to the device (e.g., GPU) before executing the forward pass?

- A) Runtime Error
- B) Vanishing Gradient Error
- C) Zero Division Error
- D) Convergence Error

Correct Answer: A) Runtime Error
Explanation: PyTorch cannot perform mathematical operations on tensors that reside on different physical hardware devices (such as CPU memory versus GPU memory). Attempting a forward pass with mismatched device locations raises a `RuntimeError` declaring device mismatch.

Question 11

Which learning rate scheduling strategy is described as smoothly decaying the learning rate throughout the training process?

- A) Linear Warm-up only
- B) Cosine Annealing
- C) Static Scheduling
- D) Ignore Index Scheduling

Correct Answer: B) Cosine Annealing
Explanation: Cosine Annealing uses a cosine curve function to gradually decrease the learning rate from a high starting point down to a minimum target value over time. This smooth decay helps the optimizer explore the loss landscape early and settle into local minima gently near the end of training.

Question 12

When implementing a custom PyTorch dataset class, which two core methods must be explicitly defined to allow the PyTorch DataLoader to automatically process and fetch samples?

- A) `__init__` and `__forward__`
- B) `__len__` and `__getitem__`
- C) `__update__` and `__collate__`
- D) `__transform__` and `__prefetch__`

Indian Institute of Technology Guwahati

Correct Answer: B) `__len__` and `__getitem__`
Explanation: The `__len__` method must return the total number of items in the dataset so the DataLoader knows its batch bounds. The `__getitem__` method defines how to retrieve and preprocess a single sample and its corresponding label by index.

Question 13

Which parameter in the PyTorch DataLoader constructor page-locks CPU memory, enabling faster and direct DMA transfers to the GPU?

- A) `persistent_workers`
- B) `drop_last`
- C) `pin_memory`
- D) `prefetch_factor`

Correct Answer: C) `pin_memory`
Explanation: Setting `pin_memory=True` allocates host (CPU) memory in a page-locked region, preventing the operating system from swapping it to disk. This enables host-to-device Direct Memory Access (DMA) transfers, significantly speeding up data movement to CUDA GPUs.

Question 14

Which of the following is the correct implementation for the missing code block inside `__getitem__` to extract a single pair of input data and its corresponding label by index?

```
import torch
from torch.utils.data import Dataset


class TextDataset(Dataset):
    def __init__(self, data, labels):
        self.data = data
        self.labels = labels


    def __len__(self):
        return len(self.data)
```

Indian Institute of Technology Guwahati

```
def __getitem__(self, index):
    # Missing Code Block
    return item, label
```

- A) item = self.data.get(index); label = self.labels.get(index)
- B) item = self.data[index]; label = self.labels[index]
- C) item = self.data.pop(index); label = self.labels.pop(index)
- D) item = self.data.next(); label = self.labels.next()

Correct Answer: B) item = self.data[index]; label = self.labels[index]

Explanation: PyTorch Datasets access underlying sequences (like lists, arrays, or tensors) using standard Python indexing syntax [index]. Using .get(), .pop(), or .next() will raise AttributeError exceptions because standard indexable sequence types do not support those methods for index lookup.

Question 15

If the entire dataset cannot be divided evenly by the chosen batch size, which DataLoader argument should be set to "True" to discard the final smaller batch?

- A) shuffle
- B) persistent_workers
- C) prefetch_factor
- D) drop_last

Correct Answer: D) drop_last

Explanation: When a dataset size isn't perfectly divisible by the batch size, the final batch will contain fewer samples than expected, which can introduce noisy gradients. Setting drop_last=True causes the DataLoader to discard this uneven trailing batch to ensure uniform batch sizes throughout training.

Indian Institute of Technology Guwahati

NPTEL Online Certification Course
Applied Accelerated Artificial Architecture
Assignment 5 : Detailed Solution
Indian Institute of Technology Guwahati

Question 1: Which high-level API is used in TensorFlow for rapid model prototyping?

Options:

A) PyTorch API

B) Keras API

C) NumPy API

D) TFX API

Answer: B) Keras API

Explanation: Keras serves as the primary high-level API within TensorFlow, providing user-friendly abstractions for constructing and training neural networks. It simplifies prototyping by streamlining layer creation, model compilation, and training execution into clean, high-level commands.

Question 2 : What decorator is used in modern TensorFlow versions to convert eager code to graph execution?

Options:

A) `@tf.graph`

B) `@tf.compile`

C) `@tf.function`

D) `@tf.eager`

Answer: C) `@tf.function`

Indian Institute of Technology Guwahati

Explanation: The `@tf.function` decorator compiles Python functions into optimized TensorFlow computation graphs using AutoGraph. This transformation provides fast graph-mode execution and hardware optimizations while allowing developers to write code imperatively.

Question 3 : How does TensorFlow's gradient tracking behavior during a standard gradient tape call differ from PyTorch's default zero_grad approach?

Options:

A) TensorFlow requires an explicit call to reset gradients.

B) TensorFlow automatically retains all calculated tape states.

C) TensorFlow clears recorded states by default after a gradient call.

D) TensorFlow lacks support for multi-loss tracking entirely.

Answer: C) TensorFlow clears recorded states by default after a gradient call.

Explanation: A standard `tf.GradientTape` automatically releases its recorded operations and memory as soon as `tape.gradient()` is called. In contrast, PyTorch accumulates gradients across operations by default until `zero_grad()` is explicitly called to reset them.

Question 4 : What parameter must be explicitly configured in a TensorFlow gradient tape to successfully evaluate multiple loss functions sequentially without throwing an error?

Options:

A) `accumulate=True`

B) `persistent=True`

C) `retain_graph=True`

D) `trace_gradients=True`

Answer: B) `persistent=True`

Explanation: Setting `persistent=True` keeps the gradient tape from releasing its recorded operations after the first gradient calculation. This enables multiple calls to

Indian Institute of Technology Guwahati

`tape.gradient()` for different loss functions, requiring manual resource cleanup using `del tape`.

Question 5: Which tensor type must be used in TensorFlow to prevent exceptions when trying to perform states or weight updates during training?

Options:

A) `tf.constant`

B) `tf.matrix`

C) `tf.variable`

D) `tf.zeros`

Answer: C) `tf.variable`

Explanation: `tf.Variable` objects represent mutable tensor states whose values can be modified in-place using update ops like `assign_sub`. Standard constants (`tf.constant`) are immutable, causing errors if modified during model weight updates.

Question 6: What is the primary advantage of executing a compiled C++ graph using tf.function over standard eager mode execution in TensorFlow 2.x?

Options:

A) Easier model debugging and line-by-line inspection.

B) Significant execution speedup for computations.

C) Direct execution of native NumPy array slices.

D) Automatic handling of non-persistent gradient tapes.

Answer: B) Significant execution speedup for computations.

Explanation: Graph compilation eliminates Python runtime overhead and enables low-level optimizations like operator fusion and constant folding. This results in significantly faster execution speeds and higher hardware utilization compared to eager evaluation.

Question 7 : How does setting "num_parallel_calls=tf.data.AUTOTUNE" optimize the data pre-processing stage in a TensorFlow input pipeline?

Indian Institute of Technology Guwahati

Options:

A) It dynamically adjusts the batch size dynamically based on memory.

B) It sequentializes data processing to save GPU bandwidth.

C) It triggers a warm-up sequence to minimize initialization time.

D) It automatically balances workloads based on CPU cores and load.

Answer: D) It automatically balances workloads based on CPU cores and load.

Explanation: Using `tf.data.AUTOTUNE` allows TensorFlow to dynamically adjust the level of parallel CPU thread execution at runtime. This removes input bottlenecks and maximizes data throughput automatically without manual thread tuning.

Question 8 : Why does a slow input data pipeline directly reduce overall GPU resource utilization during deep learning training?

Options:

A) It causes the GPU to sit idle waiting for data.

B) It throttles the C++ back-end graph template.

C) It fills the target RAM with synthetic data.

D) It reduces the multi-loss persistent gradient.

Answer: A) It causes the GPU to sit idle waiting for data.

Explanation: GPUs require a continuous pipeline of pre-processed batches to maintain high computational throughput during training steps. If CPU preprocessing is too slow, the GPU stalls waiting for incoming data, leading to underutilized compute resources.

Question 9 : During a multi-epoch training process, what behavior occurs in the second epoch if a dataset applies .map() for pre-processing without using .cache()?

Options:

A) TensorFlow re-executes the map routine calls entirely.

B) The pipeline relies on cached data from the first epoch.

C) The data pipeline automatically throws a syntax warning.

Indian Institute of Technology Guwahati

D) The preprocessing pipeline switches to a C++ graph execution.

Answer: A) TensorFlow re-executes the map routine calls entirely.

Explanation: Without `.cache()`, mapping transformations are evaluated lazily during every single epoch traversal. Using `.cache()` saves the preprocessed dataset in memory or local storage after the first epoch to prevent redundant CPU computation.

Question 10: How can you explicitly prevent unnecessary graph retracing when passing inputs with varying shapes to a @tf.function decorated function?

Options:

A) By defining a custom input_signature with None dimensions.

B) By using regular Python lists instead of TensorFlow tensors.

C) By wrapping the tensor inside a Python while loop construct.

D) By manually clearing the cache before every individual call.

Answer: A) By defining a custom input_signature with None dimensions.

Explanation: Defining dynamic dimensions as `None` within `tf.TensorSpec` allows a single compiled graph to handle variable input dimensions. This prevents TensorFlow from triggering costly graph re-compilations whenever input shapes change across calls.

Question 11 : What is the specific TensorFlow graph equivalent for a standard Python if-else branch structure when building optimized control flows?

Options:

A) `tf.while_loop`

B) `tf.print`

C) `tf.reduce_max`

D) `tf.cond`

Answer: D) `tf.cond`

Indian Institute of Technology Guwahati

Explanation: `tf.cond` builds conditional branch logic directly into the TensorFlow computational graph structure. This ensures both conditional paths can be compiled and correctly evaluated on target hardware like GPUs or TPUs.

Question 12 : During XLA's operator fusion, how are intermediate results stored to prevent High Bandwidth Memory (HBM) traffic?

Options:

A) In cached graphs

B) In global memory

C) In local registers

D) In transient tensors

Answer: C) In local registers

Explanation: XLA fuses multiple consecutive tensor operations into single hardware kernels, keeping temporary results inside ultra-fast on-chip registers. This minimizes memory bandwidth bottlenecks by avoiding unnecessary read/write operations to main HBM memory.

Question 13 : When a model is serialized using TensorFlow's native model.export method for production, what is the primary structural role of the saved "serving signature"

Options:

A) It contains optimizer configurations for resuming continuous training.

B) It encapsulates the metadata required for zero-downtime framework rollback.

C) It explicitly defines the input and output tensors required by inference APIs.

D) It maps network operations directly onto resource-constrained NPU hardware.

Answer: C) It explicitly defines the input and output tensors required by inference APIs.

Explanation: A serving signature defines the exact tensor names, shapes, and data types expected by external inference clients. This creates a clean interface contract that decouples serving environments like TensorFlow Serving from internal graph details.

Indian Institute of Technology Guwahati

Question 14 : If you want to stay strictly within the Google ecosystem and deploy a machine learning model targeting highly resource-constrained mobile apps or edge devices, which framework should you convert your SavedModel to?

Options:

A) ONNX Runtime GPU

B) TensorFlow Light

C) PyTorch TorchScript

D) TensorRT Engine

Answer: B) TensorFlow Light

Explanation: TensorFlow Lite (TFLite) is Google's dedicated framework for executing efficient inference on mobile and micro-edge devices. It applies optimizations like quantization and memory reduction to deliver low latency on embedded hardware.

Question 15 : Which specific file in a standard TensorFlow SavedModel directory is explicitly responsible for maintaining cryptographic hashes to ensure integrity verification during cross-platform model deployment?

Options:

A) `fingerprint.pb`

B) `saved_model.pb`

C) `variables.index`

D) `graph.proto`

Answer: A) `fingerprint.pb`

Explanation: The `fingerprint.pb` file stores cryptographic hashes and metadata describing the exact state of the saved model artifacts. Serving systems use it to verify model integrity and ensure consistency across cross-platform deployments.

Indian Institute of Technology Guwahati

NPTEL Online Certification Course
Applied Accelerated Artificial Architecture
Assignment 6 : Detailed Solution
Indian Institute of Technology Guwahati

Question1. What primary training objective is used during the pretraining stage of Large Language Models (LLMs)?

Options:
A. Masked token prediction
B. Next-token prediction
C. Contrastive sequence matching
D. Direct preference optimization

Correct Answer: B. Next-token prediction
Explanation:
During pretraining, LLMs learn language representations by predicting the probability distribution of the next token given preceding context.
This self-supervised autoregressive objective enables models to learn from massive amounts of unlabeled text without human annotation.

Question2. Which adaptation approach modifies external retrieved context rather than updating any internal model parameters?

Options:
A. Parameter-Efficient Fine-Tuning (PEFT)
B. Retrieval-Augmented Generation (RAG)
C. Supervised Fine-Tuning (SFT)
D. Low-Rank Adaptation (LoRA)

Correct Answer: B. Retrieval-Augmented Generation (RAG)
Explanation:
RAG fetches relevant document passages from external sources and injects them directly into the input prompt at inference time.
Because it operates solely by augmenting the prompt context, it expands model knowledge without modifying any internal model weights.

Indian Institute of Technology Guwahati

Question3. Beyond model parameters, which component often contributes significantly to memory overhead during full model fine-tuning?

Options:
A. Static input token IDs
B. Tokenizer vocabulary lookup tables
C. Fixed positional embedding matrices
D. Optimizer states and gradients

Correct Answer: D. Optimizer states and gradients
Explanation:
Full model fine-tuning requires tracking first and second momentum states along with weight gradients for every single trainable parameter.
These optimizer states and gradients combined consume several times more GPU memory than the base model parameters themselves.

Question4. What primary operation converts input text into subword units before fed into an LLM embedding layer?

Options:
A. Tokenization
B. Normalization
C. Quantization
D. De-noising

Correct Answer: A. Tokenization
Explanation:
Tokenization decomposes raw input text strings into subword units or characters based on a predefined vocabulary.
This subword tokens are mapped to discrete numerical IDs that index directly into the model's dense embedding matrix.

Question5. What is the primary benefit of utilizing Brain Float 16 (BF16) over conventional Floating Point 16 (FP16) when training large transformer models?

Options:
A. It uses fewer total bits to store numerical values.
B. It has a wider exponent range.
C. It automatically applies gradient scaling routines.
D. It can safely downcast the final output tensors.

Indian Institute of Technology Guwahati

Correct Answer: B. It has a wider exponent range.
Explanation: BF16 maintains an 8-bit exponent field matching standard FP32, effectively preventing numerical underflow and overflow during training.
This expanded dynamic range removes the need for complex loss scaling workarounds required by FP16's narrow 5-bit exponent.

Question6. How can the numerical issue of naive softmax calculations blowing up into "Not a Number" (NaN) or infinity be successfully stabilized?

Options:
A. By manually calling a blanket model.half() function.
B. By switching the target system architecture to a legacy GPU.
C. By executing the backward pass within an autocast region.
D. By subtracting the maximum value from the logits before softmax.

Correct Answer: D. By subtracting the maximum value from the logits before softmax.
Explanation:
Subtracting the maximum logit value from all input logits ensures that the maximum exponent evaluated during softmax is zero.This shift-invariant mathematical trick prevents numeric overflow during exponentiation without changing the final output probabilities.

Question7. What occurs when tiny gradients flush directly to zero during the training process, and what symptom describes this state?

Options:
A. Underflow, causing the learning process to stall.
B. Underflow, causing the loss function to hit infinity.
C. Overflow, causing the parameters to immediately vanish.
D. Overflow, causing logits to blow up before the softmax

Correct Answer: A. Underflow, causing the learning process to stall.
Explanation:
Underflow occurs when tiny gradient values fall below the representable numerical precision limit and round down to zero.Without non-zero gradient values to update model weights, backpropagation produces no parameter changes and training stalls.

Question8. Which of the following components has the same shape as the model weights when training an LLM?

Options:
A. Optimizer states

Indian Institute of Technology Guwahati

B. Gradients
C. Activations
D. Framework overhead

Correct Answer: B. Gradients
Explanation:
Gradients represent partial derivatives of the scalar loss function computed with respect to each individual model parameter.Because every trainable weight requires a corresponding gradient value, gradient tensors share an exact 1:1 structural shape with weights.

Question9. What is the primary trade-off involved when implementing gradient checkpointing in LLM training?

Options:
A. Trading compute for memory
B. Trading precision for speed
C. Trading accuracy for memory
D. Trading throughput for precision

Correct Answer: A. Trading compute for memory
Explanation:
Gradient checkpointing drastically reduces memory usage by discarding intermediate activation tensors during the forward pass.
It then recomputes these missing activations on-demand during the backward pass, increasing total FLOPs in exchange for lower VRAM footprint.

Question10. What happens to a weight update step in dynamic loss scaling if an 'inf' or 'nan' value is detected in the gradients?

Options:
A. The update step is skipped entirely.
B. The update step runs with a fixed scale.
C. Only the update of corrupted weights are skipped.
D. The model falls back to full precision.

Correct Answer: A. The update step is skipped entirely.
Explanation:
An 'inf' or 'nan' value indicates numeric overflow during backpropagation, which would permanently corrupt model parameters if applied.
The framework skips the parameter update step, reduces the scale factor for subsequent batches, and resumes training safely.

Indian Institute of Technology Guwahati

Question11. Which parameter-efficient fine-tuning technique is specifically recommended when weight updates are necessary but the memory or GPU budget is highly limited?

Options:
A. Supervised Fine-Tuning (SFT)
B. Continued Pre-training (CPT)
C. Low-Order Rank Adaptation (LoRA)
D. Zero-Shot Generation

Correct Answer: C. Low-Order Rank Adaptation (LoRA)
Explanation:
LoRA freezes pretrained model weights and introduces small, trainable low-rank decomposition matrices into transformer layers.
This reduces trainable parameters and optimizer memory by orders of magnitude, making fine-tuning feasible on limited GPU hardware.

Question12. What major limitation was observed when testing the base, pre-trained GPT-2 model using a zero-shot generation prompt?

Options:
A. It did not know how to follow the specified instruction format.
B. It failed to generate grammatically correct English sentences.
C. It was unable to process abstract concepts like networks.
D. It completely refused to output text due to policy constraints.

Correct Answer: A. It did not know how to follow the specified instruction format.
Explanation:
Base pretrained models are optimized purely for raw autoregressive text continuation rather than task or instruction execution.Without instruction fine-tuning, zero-shot prompts often cause the model to repeat or extend the prompt rather than output the answer format.

Question13. When setting up a model's weights to be frozen so that the backward pass does not update them during training, which configuration must be applied?

Options:
A. requires_grad = False
B. requires_grad = True
C. double_quantization = True
D. optimizer_state = True

Indian Institute of Technology Guwahati

Correct Answer: A. requires_grad = False

Explanation:
Setting a tensor's requires_grad attribute to False excludes it from autograd computation graph construction.This prevents autograd from computing gradients for those parameters during backpropagation, leaving the weights frozen.

Question14. During the practical implementation of LoRA, how are the weight matrices A and B typically initialized?

Options:
A. Matrix A is initialized with zeros, and Matrix B is initialized from a Gaussian distribution.
B. Matrix A is initialized from a Gaussian distribution, and Matrix B is initialized with zeros.
C. Both Matrix A and Matrix B are initialized with zeros.
D. Both Matrix A and Matrix B are drawn from a uniform float distribution.

Correct Answer: B. Matrix A is initialized from a Gaussian distribution, and Matrix B is initialized with zeros.

Explanation:
Initializing matrix B to zeros ensures that the rank update product $\Delta W = B \cdot A$ equals zero at the start of training.This guarantees that the model's baseline behavior matches original pretrained weights prior to fine-tuning updates.

Question15. If a task requires parameter updates, why is PEFT typically recommended as the first adaptation strategy instead of full fine-tuning?

Options:
A. It optimizes the entire model framework from scratch to eliminate noise.
B. It completely avoids the backward pass workflow during training.
C. It adds and trains only a few parameters to test baseline performance.
D. It guarantees higher structural throughput by using 4-bit normal floats.

Correct Answer: C. It adds and trains only a few parameters to test baseline performance.
Explanation: PEFT trains only a tiny fraction of the total model parameters, dramatically lowering compute time, VRAM, and storage requirements.
This allows rapid baseline evaluation and task adaptation before committing resources to full model fine-tuning.

Indian Institute of Technology Guwahati

NPTEL Online Certification Course
Applied Accelerated Artificial Architecture
Assignment 7: Detailed Solution
Indian Institute of Technology Guwahati

Indian Institute of Technology Guwahati

Question 1

Which parallelism strategy involves replicating the entire model on every GPU while splitting the input batch?

- Pipeline Parallelism
- Tensor Parallelism
- Data Parallelism
- Sharded Parallelism

Correct Answer: **Data Parallelism**

Explanation: In Data Parallelism, the complete model parameters are replicated across all participating GPUs, and each GPU processes a distinct partition of the batch. The computed gradients across all GPUs are then aggregated using an all-reduce operation before updating the shared weights.

Question 2

When training a model is splitted into batch wise manner where we split the input weight matrix, which parallelism strategy is required?

- Data Parallelism
- Pipeline Parallelism
- Tensor Parallelism
- Fully Sharded Data Parallel

Correct Answer: **Tensor Parallelism**

Explanation: Tensor Parallelism splits individual weight matrices and tensor operations across multiple GPUs within a single layer. This allows training massive individual layers that exceed the memory capacity of a single GPU.

Question 3

In a PyTorch DistributedDataParallel (DDP) script, what is the consequence of forgetting to call sampler.set_epoch(epoch) inside your training loop?

- The training script hangs indefinitely.
- The model runs out of memory.
- The data is shuffled identically every epoch.
- The gradients fail to synchronize.

Correct Answer: **The data is shuffled identically every epoch.**

Explanation: Calling sampler.set_epoch(epoch) updates the random seed of the DistributedSampler at the start of each epoch. Without it, the sampler reuses the exact same deterministic ordering every epoch, leading to identical batch sequences and reduced generalization.

Question 4

Which two collective communication primitives combine to form the standard Ring All-reduce operation?

- Broadcast and Reduce
- Reduce-scatter and All-gather
- All-gather and Broadcast
- Scatter and Gather

Correct Answer: **Reduce-scatter and All-gather**

Explanation: Ring All-reduce operates in two sequential phases: first, a Reduce-scatter phase aggregates gradient chunks around the ring so each GPU holds a fully reduced chunk. Then, an All-gather phase broadcasts these reduced chunks back to all GPUs so every rank receives the complete updated gradient.

Question 5

For a Ring All-reduce algorithm with N GPUs and a gradient size of M bytes, what does the per-GPU bandwidth cost "asymptotically" approach as N becomes very large?

- M bytes
- 2M bytes
- N x M bytes
- M / N bytes

Correct Answer: **2M bytes**

Indian Institute of Technology Guwahati

Explanation: Each GPU transfers $2 \times \frac{N-1}{N} \times M$ bytes of data during the combined Reduce-scatter and All-gather phases. As N becomes very large ($N \to \infty$), the factor $\frac{N-1}{N}$ approaches 1, making the total transferred data per GPU asymptotically approach $2M$ bytes.

Question 6

How does DistributedDataParallel (DDP) handle execution across GPUs to ensure true isolation?

- One thread per GPU
- One process per GPU
- One process for all GPUs
- Multiple processes per GPU

Correct Answer: **One process per GPU**

Explanation: PyTorch DDP uses a multi-process model where a dedicated Python process is spawned for each individual GPU device. This process isolation completely avoids Python's Global Interpreter Lock (GIL) bottlenecks and avoids GPU resource contention.

Question 7

When saving a checkpoint in DDP, why must you use .module.state_dict() instead of just .state_dict()?

- To avoid a 'module.' prefix on every key
- To compress the checkpoint size
- To save the optimizer state
- To sync the weights across ranks

Correct Answer: **To avoid a 'module.' prefix on every key**

Explanation: Wrapping a model in DistributedDataParallel prepends a "module." prefix to all layer names in its state_dict(). Accessing .module.state_dict() retrieves the underlying unwrapped model's state dictionary directly, ensuring key compatibility when loading checkpoints into non-DDP models.

Question 8

In a naive all-reduce implementation with N GPUs and gradient size M, what is the per-GPU communication traffic?

Indian Institute of Technology Guwahati

- O(M)
- O(N*M)
- O(M/N)
- O(N+M)

Correct Answer: **O(N*M)**

Explanation: A naive parameter-server or master-worker all-reduce implementation requires transferring the entire M-byte gradient tensor from all N workers to a central node. Consequently, total communication traffic per worker scales linearly with N, resulting in an $O(N \cdot M)$ overhead.

Question 9

During a multi-node rendezvous using c10d, what is the initial action taken by Rank 0?

- Probes for active NVLink connections
- Broadcasts the initial model weights
- Sends a unique NCCL ID to all active nodes
- Opens a TCP store and waits for connections

Correct Answer: **Opens a TCP store and waits for connections**

Explanation: During PyTorch c10d initialization, Rank 0 creates a key-value TCPStore server listening on a designated host address and port. All secondary worker ranks connect to this central TCP store during rendezvous to discover peer ranks and exchange network configuration data.

Question 10

Which of the following represents the slowest fallback option for interconnect bandwidth?

- PCIe Gen5
- InfiniBand NDR
- RoCEv2 Ethernet
- TCP sockets

Correct Answer: **TCP sockets**

Explanation: Standard TCP sockets rely on CPU software stack processing and standard network protocols, introducing significant latency and lower bandwidth. High-performance interconnects like InfiniBand NDR, RoCEv2, and PCIe Gen5 utilize hardware direct memory access (RDMA/GPUDirect) to bypass CPU software bottlenecks.

Indian Institute of Technology Guwahati

Question 11

What is the primary characteristic of "weak scaling" in distributed training?

- Training a bigger problem with constant wall time.
- Finishing a fixed job faster with more GPUs.
- Reducing the communication overhead to zero.
- Minimizing the batch size per GPU.

Correct Answer: **Training a bigger problem with constant wall time.**

Explanation: Weak scaling evaluates execution efficiency when the workload per GPU (local batch size) is kept fixed as more GPUs are added. The goal is to scale up the total model capacity and dataset size while keeping total execution time constant.

Question 12

According to the PyTorch profiler breakdown, what is the recommended fix for addressing "load imbalance" during training?

- Pinning processes to healthy GPUs.
- Using fewer Python callbacks.
- Ensuring same-shape inputs (pad or drop).
- Tuning NCCL bucket sizes.

Correct Answer: **Ensuring same-shape inputs (pad or drop).**

Explanation: Load imbalance occurs when different GPUs process variable sequence lengths or batch sizes, forcing faster ranks to wait idly at synchronization points for slower ranks. Standardizing input tensor shapes across all ranks via padding or dropping residual samples eliminates execution variance.

Question 13

How is scaling efficiency mathematically calculated for an N-GPU training run?

- throughput(N) / throughput(1)
- N * throughput(N) / throughput(1)
- throughput(N) / (N * throughput(1))
- throughput(1) / (N * throughput(N))

Correct Answer: **throughput(N) / (N * throughput(1))**

Explanation: Scaling efficiency measures how close a distributed cluster comes to ideal linear speedup by dividing total throughput on N GPUs by N times single-GPU throughput.

Indian Institute of Technology Guwahati

A value of 1.0 (100%) indicates perfect linear scaling without communication overhead losses.

Question 14

Which optimization lever is considered the dominant factor for improving communication efficiency?

- Mixed precision (AMP)
- Hierarchical all-reduce
- Gradient compression
- Larger per-GPU batch + gradient accumulation

Correct Answer: **Larger per-GPU batch + gradient accumulation**

Explanation: Increasing local batch sizes and accumulating gradients over multiple micro-batches decreases the frequency of inter-GPU communication calls. Extending computation time relative to networking time drastically reduces communication bottlenecks and boosts overall GPU compute utilization.

Question 15

If you are training a 70B parameter model where the overall memory is very tight, but individual layers still fit perfectly on a GPU, which tool is recommended?

- Fully Sharded Data Parallel (FSDP)
- DistributedDataParallel (DDP)
- Tensor Parallelism (Megatron)
- 3D Parallelism (Megatron-DeepSpeed)

Correct Answer: **Fully Sharded Data Parallel (FSDP)**

Explanation: FSDP shards model parameters, gradients, and optimizer states across data-parallel ranks, gathering parameters layer-by-layer only when needed during forward and backward passes. This eliminates parameter duplication while allowing large models whose individual layers fit into GPU memory to train efficiently.

Indian Institute of Technology Guwahati

NPTEL Online Certification Course
Applied Accelerated Artificial Architecture
Assignment 8: Detailed Solution
Indian Institute of Technology Guwahati

Indian Institute of Technology Guwahati

Question 1: According to the linear scaling rule, if the baseline learning rate is 0.075 at a batch size of 512, what should the new learning rate be if the batch size is increased to 4096?

- 0.2
- 0.6
- 0.75
- 0.8

Correct Answer: 0.6

Explanation: The linear scaling rule scales the learning rate proportionally with the batch size increase (4096/512=8). Multiplying the baseline learning rate of 0.075 by 8 yields 0.6.

Question 2: What is defined as the point past which increasing the batch size further stops providing proportional returns in training efficiency?

- Effective batch size
- Baseline batch size
- Divergence batch size
- Critical batch size

Correct Answer: Critical batch size

Explanation: The critical batch size marks the inflection point where hardware scaling efficiency drops and further batch size increases yield diminishing returns in gradient noise reduction.

Question 3: What happens during a training "warm-up" phase to prevent early divergence?

- The learning rate is immediately set to its maximum value.
- The learning rate is linearly ramped up from near zero to the full value.
- The batch size is gradually decreased over several iterations.
- The gradients are completely cleared for the first 500 steps.

Correct Answer: The learning rate is linearly ramped up from near zero to the full value.

Explanation: Starting with a very small learning rate and gradually increasing it prevents large, destabilizing gradient updates when the model parameters are still random and unstable.

Question 4: Why does a single slow GPU slow down the entire multi-GPU training process?

- Because all GPUs must finish a step and synchronize before moving to the next.
- Because the system automatically reduces the overall batch size.
- Because the slow GPU forces a roll back to the last checkpoint.
- Because it triggers an automatic switch from Adam to the LAMB optimizer.

Correct Answer: Because all GPUs must finish a step and synchronize before moving to the next.

Explanation: Data-parallel synchronous training relies on barrier synchronization at each iteration, meaning fast GPUs must idle and wait for the slowest worker to finish its gradient computation.

Question 5: Which term describes a system's ability to keep working or recover properly when a part of it fails?

- Sharding
- Elastic training
- Fault tolerance
- Rendezvous

Correct Answer: Fault tolerance

Explanation: Fault tolerance mechanisms ensure that distributed systems can detect node failures, preserve progress via checkpoints, and recover without restarting from scratch.

Question 6: If a training script saves a checkpoint at step 700 and a worker's GPU crashes at step 743, what happens when torchrun restarts the job?

- Training will start completely over from step 0.
- Training skips ahead to step 800 to avoid the corrupted block.
- Training continues exactly from step 743 without losing any work.
- Training resumes from step 700, losing 43 steps of work.

Correct Answer: Training resumes from step 700, losing 43 steps of work.

Explanation: Since checkpoints are saved periodically at discrete intervals, restarting after a crash rolls back execution to the most recent successfully saved state.

Question 7: What is a major advantage of utilizing an asynchronous save (async checkpoint) over a synchronous save?

- It splits the dataset into smaller, easier-to-manage batches.

Indian Institute of Technology Guwahati

- It automatically reduces the learning rate to prevent memory fragmentation.
- It requires no background processing or coordination among workers.
- It hands the save to a background process so foreground training keeps running.

Correct Answer: It hands the save to a background process so foreground training keeps running.

Explanation: Asynchronous checkpointing serializes model weights in a separate background thread or process, preventing the main GPU training loop from stalling during disk I/O.

Question 8: Why is it highly discouraged to ship raw optimizer states along with a model for inference?

- It increases model size by 4x to 5x without aiding inference
- It restricts the model to cross-platform frameworks
- It prevents the framework from utilizing numeric formats
- It completely stops the training progression

Correct Answer: It increases model size by 4x to 5x without aiding inference

Explanation: Optimizer states like momentum and variance buffers are strictly required for backpropagation updates and gradient optimization, serving zero purpose during forward-pass inference.

Question 9: In a model registry, what status is given to a model version that is currently live and serving real user traffic?

- Staging
- Production
- Archived
- None

Correct Answer: Production

Explanation: The production tag designates the verified model artifact currently deployed in live operational environments to handle real client queries.

Question 10: Which deployment strategy involves sending a small slice of traffic to a new model version before fully switching over?

- Rolling update
- Blue-green deployment
- Canary deployment
- Shadow rollout

Correct Answer: Canary deployment

Indian Institute of Technology Guwahati

Explanation: Canary deployments route a fractional percentage of live production traffic to a new model version to test stability and real-world behavior safely.

Question 11: In Kubernetes, what mechanism is used to check the health and status of a container?

- Rollback
- Artifact
- Probe
- Budget

Correct Answer: Probe

Explanation: Kubernetes uses liveness, readiness, and startup probes to periodically assess container health and determine if traffic should be routed to it.

Question 12: What is the default Kubernetes rollout strategy for most stateless services?

- Blue-green deployment
- Rolling update
- Canary deployment
- Shadow rollout

Correct Answer: Rolling update

Explanation: Rolling updates incrementally replace old pods with new pods one by one or in small batches, ensuring zero downtime during application deployments.

Question 13: What occurs during a linear learning rate warm-up over the first 10% of training steps?

- The learning rate increases gradually to its peak value.
- The learning rate drops instantly to zero.
- The batch size decreases at a steady rate.
- The model updates stop entirely for those steps.

Correct Answer: The learning rate increases gradually to its peak value.

Explanation: Warm-up safely scales the learning rate from zero up to the target learning rate, preventing explosive gradient updates at the start of training.

Question 14: Why does large batch training typically underperform if a high learning rate is immediately used without a warm-up period?

- It experiences too many optimization steps per epoch.

Indian Institute of Technology Guwahati

- It suffers from fewer optimization steps per epoch.
- The dataset size automatically shrinks.
- The GPU memory completely runs out.

Correct Answer: It suffers from fewer optimization steps per epoch.

Explanation: Large batches lead to fewer total weight updates per epoch, meaning high initial learning rates can cause unstable gradient trajectories before the optimizer stabilizes.

Question 15: Which specific deployment target is the TorchScript (.ts) model packaging format typically used for?

- Hugging Face Transformers
- ONNX Runtime engines
- Cloud-hosted TensorRT servers
- Mobile or standalone C++ applications

Correct Answer: Mobile or standalone C++ applications

Explanation: TorchScript serializes PyTorch models into a static graph format, enabling execution in non-Python runtime environments like mobile apps and C++ backends.

Indian Institute of Technology Guwahati

NPTEL Online Certification Course
Applied Accelerated Artificial Architecture
Assignment 9 : Detailed Solution
Indian Institute of Technology Guwahati

Q1. Which graph capture mechanism cannot handle dynamic control flow (such as if-else statements) because it is highly tensor shape specific?

- torch.export
- torch.jit.script
- torch.jit.trace
- torch.compile

Correct Answer: **torch.jit.trace**

Explanation: `torch.jit.trace` records operations based on a single execution pass with example inputs. Therefore, it silently ignores alternative control flow paths not taken during that specific trace, making it unsuitable for highly dynamic logic.

Q2. What is the recommended modern API for capturing graphs with dynamic shape support in PyTorch versions 2.x and later?

- torch.jit.trace
- torch.export
- torch.jit.script
- torch.cuda.graph

Correct Answer: **torch.export**

Explanation: Introduced in PyTorch 2.0, `torch.export` is the standard ahead-of-time (AOT) compilation tool designed to capture full graph structures. It reliably handles dynamic shapes and produces a standardized ExportedProgram suitable for stable deployment.

Q3. During a transformer forward pass execution on an NVIDIA GPU at batch size 1, what primary issue causes the GPU to remain idle while waiting for the next kernel call?

- Matrix multiplication latency
- CPU launch overhead
- Flash attention tiling

Indian Institute of Technology Guwahati

- GPU memory fragmentation

Correct Answer: **CPU launch overhead**

Explanation: At a batch size of 1, the computational workload per kernel is so small that the GPU finishes executing tasks faster than the CPU can queue them up. This bottleneck leaves the GPU idle while waiting for the CPU to dispatch subsequent instructions.

Q4. What are the graph nodes represented as in an ONNX graph?

- Edge devices
- Core weights
- Tensor edges
- Operators

Correct Answer: **Operators**

Explanation: In the ONNX (Open Neural Network Exchange) computational graph model, the individual nodes represent mathematical or functional operators (such as Convolution, ReLU, or MatMul). The edges that connect these nodes represent the tensors flowing between the operations.

Q5. Why can an execution provider fallback hurt the overall performance during model serving?

- It disables the operator fusions permanently.
- It causes a reduction in model parameters.
- It introduces data transfer and synchronization overhead.
- It changes static shapes into symbolic ones.

Correct Answer: **It introduces data transfer and synchronization overhead.**

Explanation: When a hardware accelerator (like a GPU) cannot execute a specific node, the ONNX Runtime falls back to the CPU. This forces data to be actively transferred between the device memory and the host memory, creating significant latency and synchronization delays.

Q6. In which scenario is the ONNX runtime considered an ideal choice for model deployment?

- When maximizing the throughput of a specific specialized hardware target
- When deploying cross-platform models across various target environments
- When executing highly customized neural networks on restricted single-node systems
- When managing complex training loops and backpropagation tasks

Indian Institute of Technology Guwahati

Correct Answer: **When deploying cross-platform models across various target environments**

Explanation: ONNX Runtime is inherently designed for wide interoperability and portability. It allows a single exported model to be deployed seamlessly across different operating systems, hardware architectures, and accelerators using various execution providers.

Q7. What optimization happens during the "tactic selection" phase of TensorRT's pipeline?

- It permanently fusions layers together.
- It selects the fastest kernel for an operation.
- It reduces the memory fragmentation.
- It adjusts the dynamic shape range boundaries.

Correct Answer: **It selects the fastest kernel for an operation.**

Explanation: During tactic selection, TensorRT automatically profiles multiple hardware-specific kernel implementations (tactics) for a given layer. It benchmarks these candidate kernels directly on the target GPU to choose the one that delivers the fastest execution time.

Q8. Which specific optimization technique is implemented in TensorRT-LLM to reduce memory fragmentation and improve concurrency?

- Shape specialization profiles
- Operator kernel serialization
- Paging for large KV caches
- Explicit data type fallback

Correct Answer: **Paging for large KV caches**

Explanation: TensorRT-LLM utilizes PagedAttention, which treats the Key-Value (KV) cache like virtual memory mapped into non-contiguous blocks (pages). This optimization essentially eliminates memory fragmentation and allows the system to process much larger batches concurrently.

Q9. Why is engine building in TensorRT considered a decoupled and expensive process?

- It repeatedly executes complete backpropagation steps.
- It downloads pre-optimized architectures over the network.
- It converts symbolic layouts to static arrays dynamically.
- It explores and tests multiple candidate kernels.

Correct Answer: **It explores and tests multiple candidate kernels.**

Indian Institute of Technology Guwahati

Explanation: The engine building phase involves an exhaustive local search where TensorRT profiles hundreds of potential kernel fusions and implementations on the specific target hardware. This benchmarking process takes considerable

Q10. What does the parameter "zero point" represent in the context of quantization?

- The maximum integer value in a scale
- The margin of error after reconstruction
- The integer code representing real zero
- The spacing between representable labels

Correct Answer: The integer code representing real zero

Explanation: In asymmetric quantization, the "zero point" is the quantized integer value that maps exactly to the real-world floating-point value of 0.0. This guarantees that operations like zero-padding, which are common in neural networks, introduce no approximation errors.

Q11. What is a key difference between Post-Training Quantization (PTQ) and Quantization-Aware Training (QAT)?

- PTQ is much more expensive to run than QAT.
- QAT simulates quantization during the training or fine-tuning stage.
- PTQ provides a higher baseline accuracy than QAT.
- QAT requires storing less metadata than PTQ.

Correct Answer: QAT simulates quantization during the training or fine-tuning stage.

Explanation: While PTQ directly converts a pre-trained model's weights without any retraining, QAT integrates fake quantization nodes into the computational graph during the training process itself. This allows the model to learn and adapt to quantization errors, resulting in higher accuracy.

Q12. Why is large language model (LLM) quantization trickier compared to other traditional deep learning models?

- LLM weights and activations can contain outlier channels.
- Traditional deep learning models do not use activation clipping.
- LLMs cannot utilize approximate second-order information.
- LLM hardware architectures lack support for matrix multiplication.

Correct Answer: LLM weights and activations can contain outlier channels.

Explanation: Large Language Models frequently exhibit extreme outlier values in specific activation channels that are crucial for accuracy. Quantizing these dramatic outliers with

Indian Institute of Technology Guwahati

standard broad scales drastically degrades model performance without specialized scaling techniques.

Q13. Which technique utilizes approximate second-order information (usually the Hessian matrix) to compensate for layer-wise quantization errors?

- Static PTQ
- AWQ
- GPTQ
- Dynamic PTQ

Correct Answer: GPTQ

Explanation: GPTQ (Generative Pre-trained Transformer Quantization) uses an optimal brain quantization approach that relies on the inverse Hessian matrix. As it iteratively quantizes weights, it adjusts the remaining unquantized weights to explicitly compensate for the introduced error.

Q14. The Activation-error Weight Quantization (AWQ) technique primarily relies on calibration data to identify which of the following?

- Backpropagation thresholds
- Outlier hardware nodes
- Linear regression targets
- Salient activation channels

Correct Answer: Salient activation channels

Explanation: AWQ observes a small set of calibration data to determine which activation channels are the most "salient" or important to the model's overall output. It then protects the corresponding weights associated with these salient channels from aggressive quantization.

Q15. What is a key benefit of using group-wise scaling over choosing a single scale for an entire matrix?

- It avoids the need to store any metadata.
- It minimizes the quantization error.
- It completely eliminates the need for suitable kernels.
- It drastically reduces the runtime memory load.

Correct Answer: It minimizes the quantization error.

Explanation: Group-wise scaling divides a large weight matrix into smaller functional blocks (groups) and computes an independent scale and zero point for each block. This

Indian Institute of Technology Guwahati

localized precision significantly reduces the overall quantization error compared to using a single scale for the entire tensor.

Indian Institute of Technology Guwahati

NPTEL Online Certification Course
Applied Accelerated Artificial Architecture
Assignment 10 : Detailed Solution
Indian Institute of Technology Guwahati

Q1. Which operation is considered the most expensive in a distributed data processing pipeline because it involves regrouping and moving data across workers?

- A. Evaluation
- B. Shuffle
- C. Partitioning
- D. Lineage

Correct Answer: Shuffle
Explanation: Shuffling redistributes data across different partitions, which heavily utilizes network bandwidth, disk I/O, and data serialization. This massive movement of data between worker nodes naturally makes it the most resource-intensive operation in distributed computing.

Q2. Which of the following best defines the concept of "data lineage" within a distributed data engineering architecture?

- A. A dynamic layout mechanism that optimizes physical disk storage distribution.
- B. A deterministic profile that calculates network latency between operational workers.
- C. A record of data derivation that facilitates state reconstruction in the event of failure.
- D. An immediate evaluation strategy that executes instruction blocks line by line.

Correct Answer: A record of data derivation that facilitates state reconstruction in the event of failure.
Explanation: Data lineage tracks the exact sequence of transformations applied to a base dataset, creating a logical execution plan. If a node fails, the system uses this lineage graph to recompute only the lost partitions from the original data instead of restarting the entire pipeline.

Q3. How does a tool like Spark or Dask handle instruction execution differently compared to Pandas?

- A. It relies heavily on strict static allocation
- B. It executes each individual line immediately

Indian Institute of Technology Guwahati

- C. It uses a highly centralized row range lookup
- D. It employs an optimization via lazy evaluation

Correct Answer: It employs an optimization via lazy evaluation
Explanation: Unlike Pandas, which executes operations eagerly line-by-line, frameworks like Spark build a directed acyclic graph (DAG) of transformations. Execution only begins when an action is called, allowing the engine to optimize the entire query plan for maximum efficiency.

Q4. In distributed data systems, how does a scheduler notice that a worker has unexpectedly failed mid-group?

- A. It detects that the worker stopped heartbeating
- B. It scans the primary key for missing data entries
- C. It identifies that a local task graph has ended
- D. It encounters an automatically forced row range

Correct Answer: It detects that the worker stopped heartbeating
Explanation: Worker nodes periodically send quick "heartbeat" signals over the network to the central cluster manager to confirm they are active. If the scheduler stops receiving these heartbeats within a specific timeout window, it assumes the worker has crashed and reassigns its tasks.

Q5. Which partitioning scheme is ideal for a clickstream group by user ID to achieve zero shuffle?

- A. Hashing
- B. Salting
- C. Row range
- D. Range date

Correct Answer: Hashing
Explanation: By hashing the user ID, all records belonging to the same user will consistently map to the exact same partition. When a "group by" is performed on this ID, the data is already localized on the correct node, eliminating the need to shuffle data across the network.

Q6. Which framework is nearly identical to Pandas and best suited for tabular ETL ranging from 10 GB to 10 terabytes?

- A. A stateless function
- B. A cloud-managed storage pod
- C. A data partitioning scheduler
- D. A stateful Python object

Correct Answer: A stateful Python object
Explanation: While phrasing is unconventional, distributed architectures for tabular ETL

Indian Institute of Technology Guwahati

(like Ray or Dask) rely on stateful Python objects (Actors) to scale processing. These tools maintain state across nodes to handle datasets that exceed single-machine memory constraints.

Q7. Which deployment model allows for dynamic autoscaling of pods when working with Dask?

- A. Local cluster
- B. Kubernetes cluster
- C. Cloud-managed service
- D. Single core machine

Correct Answer: Kubernetes cluster
Explanation: Kubernetes natively supports dynamic resource allocation, allowing distributed frameworks like Dask to seamlessly spin up or tear down worker pods based on the real-time workload. This elastic autoscaling ensures optimal resource utilization and cost efficiency.

Q8. What is a common mistake when managing frameworks, which leads to poor GPU utilization?

- A. Using Ray tune for model serving
- B. Choosing Dask for GPU inference
- C. Deploying Ray tasks on Kubernetes
- D. Applying Pandas APIs to Dask arrays

Correct Answer: Choosing Dask for GPU inference
Explanation: Dask is primarily designed for distributed task scheduling and data processing, not for managing the strict memory complexities of GPU model inference. Utilizing specialized serving frameworks is necessary to achieve high throughput and avoid severe GPU bottlenecks.

Q9. What is the purpose of "salting" in the context of handling data skew?

- A. It deletes duplicate rows from the partition.
- B. It caches data frames directly into the CPU memory.
- C. It adds a random value to a skewed key to spread rows.
- D. It converts a Python user-defined function into SQL.

Correct Answer: It adds a random value to a skewed key to spread rows.
Explanation: Data skew occurs when one key has vastly more records than others, overloading a single partition. Salting adds a randomized prefix or suffix to this problematic key, artificially breaking the massive group into smaller sub-keys that distribute evenly across nodes.

Q10. For which size of data sets will GPUs not provide any processing benefit when using CUDF?

Indian Institute of Technology Guwahati

- A. Less than 100k rows
- B. Around 10 million rows
- C. More than 50 million rows
- D. Petabyte-scale data sets

Correct Answer: Less than 100k rows
Explanation: For very small datasets, the overhead time spent transferring data from the CPU to the GPU's memory (the PCIe bottleneck) takes longer than the actual computation. CPUs are already fast enough to process sub-100k rows instantly, making GPU acceleration counterproductive.

Q11. Which component in Spark automatically rewrites queries to optimize performance and speed?

- A. Executor
- B. Delta Lake
- C. Cluster Manager
- D. Catalyst Optimizer

Correct Answer: Catalyst Optimizer
Explanation: The Catalyst Optimizer is Spark's advanced query planning engine that parses SQL and DataFrame operations into an optimized physical execution plan. It applies smart rules like filter pushdown and column pruning to heavily reduce data scanning times.

Q12. What common mistake in Spark causes the driver to run out of memory and crash?

- A. Calling .collect() on a huge data set
- B. Utilizing Adaptive Query Execution
- C. Writing data directly to the storage
- D. Processing data sets with small keys

Correct Answer: Calling .collect() on a huge data set
Explanation: The `.collect()` operation forces all distributed worker nodes to send their entire partitioned data back to a single driver node. If the resulting dataset exceeds the driver node's allocated memory capacity, it throws an OutOfMemory (OOM) error and crashes.

Q13. Which of the following is defined as a discrete, timestamped record of something that actually happened in a system?

- A. Metric
- B. Log
- C. Trace
- D. SLO

Indian Institute of Technology Guwahati

Correct Answer: Log
Explanation: A log is an immutable text record detailing a specific event or transaction that occurred within an application at an exact moment in time. Unlike aggregated metrics, logs provide granular context regarding errors and systemic behaviors for debugging.

Q14. In the provided ETL troubleshooting scenario, what was diagnosed as the root cause for the runtime jumping from 40 minutes to 3 hours?

- A. Disk storage failure
- B. Data skew from a new bot account
- C. Network packet loss
- D. Wrong schema definition

Correct Answer: Data skew from a new bot account
Explanation: A sudden, dramatic increase in ETL runtime is frequently caused by data skew, where a massive influx of anomalous records (like spam bot activity) maps to a single partition. This creates a "straggler task" where one node bottlenecks the entire pipeline.

Q15. When investigating a data pipeline bottleneck, in what specific order should you analyze the three complimentary signals?

- A. Traces, Metrics, Logs
- B. Metrics, Traces, Logs
- C. Logs, Metrics, Traces
- D. Traces, Logs, Metrics

Correct Answer: Logs, Metrics, Traces
Explanation: Analyzing logs first allows engineers to identify exact error messages or immediate anomalies at the timestamp of the bottleneck. Once isolated, metrics can quantify the system's overall resource health, and traces map the request's path to find the specific latency source.

Indian Institute of Technology Guwahati